

1. Principios de Prueba

- **Las pruebas dependen del contexto:** No se prueba igual una aplicación móvil que un sistema bancario; la estrategia debe adaptarse al riesgo y al modelo de desarrollo (SDLC).
 - **Agrupación de defectos:** Una pequeña cantidad de módulos suele contener la mayoría de los defectos detectados; por ello, las pruebas deben priorizar esas áreas.
 - **Paradoja del pesticida:** Si se repiten las mismas pruebas, dejarán de encontrar defectos nuevos; es necesario revisar y actualizar las suites de pruebas regularmente.
 - **La ausencia de errores es una falacia:** Encontrar y corregir todos los defectos no garantiza el éxito si el sistema no cumple con las necesidades del usuario o sus expectativas de usabilidad.
-

2. Niveles y Tipos de Prueba

- **Niveles de Prueba:** Cada nivel tiene objetivos específicos para detectar diferentes tipos de fallos:
 - **Componente (Unitarias):** Verifica módulos o clases de forma aislada.
 - **Integración:** Detecta defectos en las interfaces y la interacción entre componentes.
 - **Sistema:** Valida el producto completo en un entorno similar a producción, incluyendo requisitos funcionales y no funcionales.
 - **Aceptación:** Confirma que el sistema está listo para el uso operativo y cumple con los requisitos del negocio.
 - **Pruebas Funcionales vs. Estructurales:**
 - Las **funcionales** validan el comportamiento externo ("qué hace el sistema").
 - Las **estructurales** (caja blanca) se centran en las rutas internas del código.
-

3. Pruebas en Modelos Ágiles y DevOps

- **Pruebas Continuas:** En entornos DevOps, los niveles de prueba se difuminan en un flujo continuo gracias a la automatización y la retroalimentación rápida.
 - **Mentalidad de Equipo:** En Ágil, el control de calidad está integrado en el equipo multifuncional, no es una fase aislada al final.
 - **Documentación Viva:** Los casos de prueba no deben ser estáticos; se deben refactorizar y actualizar continuamente junto con las historias de usuario.
 - **Pruebas Tempranas:** Los evaluadores deben participar desde el refinamiento del backlog para identificar ambigüedades antes de que se escriba el código.
-

4. Gestión y Análisis de Riesgos

- **Pruebas Basadas en Riesgos:** Debido a que las pruebas exhaustivas son imposibles, se utiliza el análisis de riesgos para priorizar qué probar primero y con qué profundidad.

- **Criterios de Salida:** Es vital definir condiciones objetivas y medibles para saber cuándo dejar de probar, basándose en la cobertura y la tolerancia al riesgo.
 - **Trazabilidad:** Mantener un vínculo claro entre requisitos, casos de prueba y defectos permite asegurar que todo lo solicitado ha sido verificado y facilita el análisis de impacto ante cambios.
-

5. Entornos y Datos de Prueba

- **Realismo del Entorno:** El entorno de pruebas debe reflejar la producción lo más fielmente posible para asegurar que los resultados sean confiables.
- **Gestión de Datos:** Se deben usar datos realistas (sintéticos o anonimizados) que simulen la escala de producción para detectar problemas de rendimiento o volumen que no aparecen con datos pequeños.
- **Infraestructura como Código (IaC):** En la entrega continua, es una buena práctica aprovisionar entornos de forma automatizada y consistente mediante código.